Project 01 — Interactive Voice Agent on TensorRT-LLM

GPU server · real-time speech-to-speech · sub-second turn-taking

Contents

Project 01 — Interactive Voice Agent (TensorRT-LLM)

Read `00-shared-requirements.md` first. Everything in it applies here.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **TensorRT-LLM on Triton serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §6 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a "Say this" line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest `source` badge — except Improvement, which is never seeded.

8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §7. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §7a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §8 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §12 of this document plus the shared deliverables checklist.

---

1. Brief

Build a **real-time voice agent** that a caller can interview, interrupt and argue with — a spoken support line, not a transcription demo. A user speaks into the browser, the agent answers in speech, and the exchange feels like a phone call rather than a walkie-talkie.

The engineering target that defines the project:

> **Under 800 ms from the user stopping speaking to the first audio of the reply**, measured end to end at p95, with barge-in supported throughout.

That single number drives every decision below. Above roughly one second, people start talking over the agent because they assume it did not hear them; the conversation collapses. This is not a comfort target, it is the threshold at which the product works at all.

Target user: a company replacing an IVR phone tree with something that can actually answer questions, or adding a voice channel to an existing support desk.

---

2. Why TensorRT-LLM for this workload

Voice is the hardest latency case in LLM serving, and it is where TensorRT-LLM's specific advantages are decisive rather than marginal:

- **Time to first token *is* the product.** In a chat UI, TTFT is a comfort metric — text can stream in leisurely. In voice, TTFT gates when the first audio chunk can be synthesised, so it sits directly in the turn-latency budget. TensorRT-LLM's ahead-of-time kernel compilation, fused attention and layer fusion attack exactly this number.
- **In-flight (continuous) batching** keeps the GPU busy across concurrent calls without making any single caller wait for a batch to fill. A voice workload is bursty and latency-critical simultaneously — the worst case for static batching.
- **Paged KV cache** lets many concurrent calls share GPU memory efficiently. Conversations are long and mostly idle; without paging, each call would reserve its worst-case context and concurrency would collapse.
- **FP8 on Hopper/Ada** roughly halves weight memory and improves throughput, and on a latency-bound workload the memory freed becomes concurrency headroom.
- **Triton Inference Server integration** gives a production serving layer with request cancellation — which matters enormously here, because **barge-in means abandoning generation mid-stream**, constantly.

The honest counterpoint, state it in the README: TensorRT-LLM requires an engine-build step per model, per precision, per GPU architecture. You trade deployment flexibility for latency. For this workload that trade is correct; for a system that swaps models weekly it would not be.

---

3. Model and compression

Stage	Choice	Notes
ASR	`distil-whisper/distil-large-v3` or Parakeet	Streaming, partial hypotheses
LLM	`Llama-3.1-8B-Instruct` or `Qwen2.5-7B-Instruct`	FP8 via TensorRT-LLM quantization toolkit
TTS	Piper, Kokoro, or XTTS-v2	Must support **streaming chunk output**

Non-negotiable TTS property: it must emit audio in chunks as text arrives, not after a complete sentence. A TTS that waits for the full reply adds its entire synthesis time to the turn budget and makes the target unreachable regardless of how fast the LLM is.

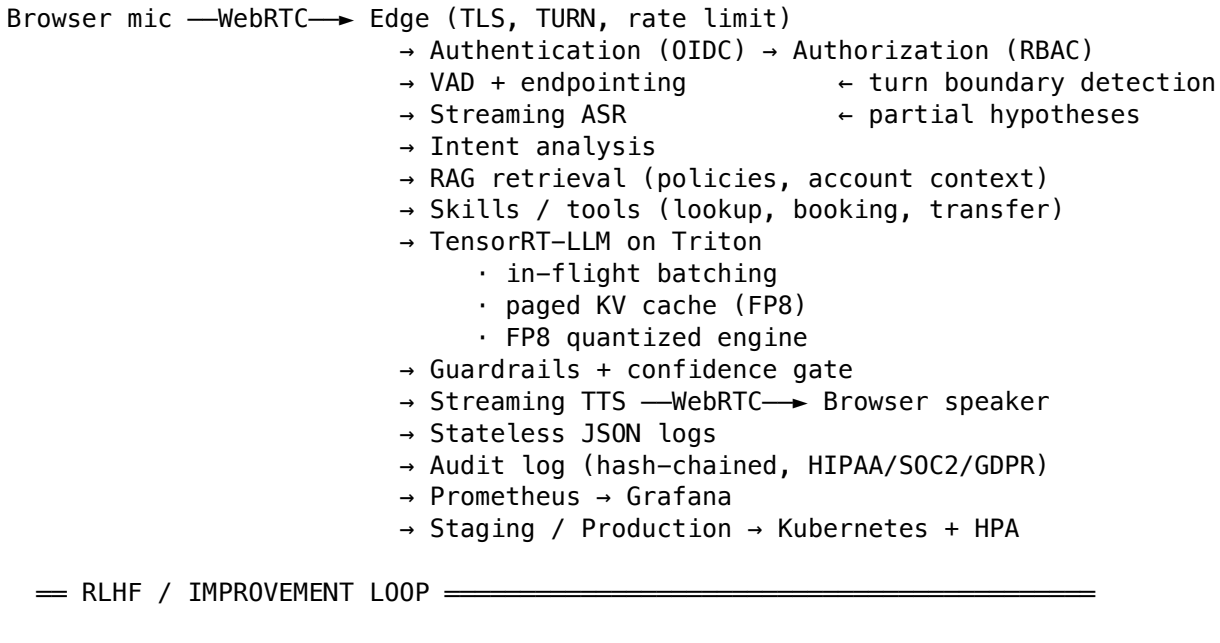
Compression spec - Build the TensorRT-LLM engine with FP8 weights and FP8 KV cache on Hopper/Ada (H100, L40S). Fall back to INT4 AWQ on Ampere. - Record weights before and after, and state what the freed memory buys — here it is concurrent calls per GPU, which is the unit the client is paying for. - Enable in-flight batching and paged KV in the engine config.

Quality gate - Academic: instruction-following benchmark, gated as a delta against the uncompressed baseline. - Domain suite: a held-out set of spoken support exchanges scored on whether the answer is correct, whether it stays inside supplied policy, and whether it correctly escalates when it does not know. - **Voice-specific:** measure word error rate through the full ASR → LLM → TTS loop, not on clean text. Compression damage shows up differently when the input is already noisy from ASR.

---

4. Architecture

The full chain from `00-shared-requirements.md` §3, with these specifics:



Design points worth building deliberately, because they are what the demo shows:

- **Barge-in.** When the VAD detects the user speaking while the agent is talking, cancel the in-flight LLM request *and* stop TTS playback immediately. Triton supports request cancellation; use it. Without cancellation the GPU keeps generating tokens nobody will hear, which is both a latency and a cost problem.
- **Endpointing is a product decision, not a default.** How long a silence means "your turn" determines whether the agent feels attentive or impatient. Make it configurable and show it in the Product tab.
- **Speculative first response.** Optionally begin generating on the ASR partial hypothesis and discard if the final transcript diverges. This buys real latency and costs wasted tokens — measure both before committing.

---

5. Frontend

Per §2 of the shared requirements. Project-specific detail:

Product tab — a call interface: push to talk or open mic, live waveform, live partial transcript, the agent's reply as both audio and text, and a **turn-latency readout broken into stages** (VAD □ ASR □ LLM TTFT □ TTS first chunk). That breakdown is the single most persuasive thing on screen for a technical buyer, because it shows exactly where the budget goes.

Configuration: agent persona, company policies, escalation rules, endpointing silence threshold, voice selection.

Architecture tab — 3D graph. Logos: NVIDIA, TensorRT, Triton, Kubernetes, Prometheus, Grafana, PostgreSQL. The nodes needing the strongest cost/quality copy are **in-flight batching**, **paged KV cache**, **FP8 engine** and **barge-in cancellation**.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §6 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

---

6. Architecture graph: node inventory

These are the stage cards for this project's Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent's card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a "Say this" line on each.

#	Stage card	Colour role	Expands into
0	**Caller**	neutral	— (the person on the line)
1	**Voice Client**	teal	Mic capture, WebRTC transport, playback, barge-in detector
2	**Edge & Security**	amber	TLS + TURN, rate limiting, authentication (OIDC), authorization (RBAC), spoken-injection detection

#	Stage card	Colour role	Expands into
3	**Turn Detection**	violet	VAD, endpointing threshold, streaming ASR, partial hypotheses
4	**Context & Skills**	violet	Intent analysis, RAG retrieval, account lookup, booking/transfer tools, prompt compiler, guardrails
5	**TensorRT-LLM Engine**	orange	**Llama-3.1-8B-Instruct (FP8)**, in-flight batching, paged KV cache (FP8), fused kernels & AOT engine build, request cancellation
6	**Speech Output**	teal	Streaming TTS, chunked synthesis, playback buffer
7	**Database**	blue	PostgreSQL, transcript store, model/engine registry
8	**Monitoring & Audit**	magenta	Prometheus, Grafana, hash-chained audit log, consent records, structured logging, alerting
9	**Improvement Loop**	green	Turn rating, A/B voice comparison, preference dataset, answer scoring, DPO fine-tune, promotion gate `lm_eval`, `GuideLLM`
10	**Platform**	grey	Kubernetes, HPA on active-call count, staging production

The card that carries the commercial argument is **TensorRT-LLM Engine**. Its `request cancellation` child is the one most people have never thought about: barge-in means abandoning generation constantly, and without cancellation the GPU keeps producing tokens nobody will hear — a latency *and* a cost problem.

---

7. The improvement loop for this project

`00-shared-requirements.md` §5a applies in full. What is specific here:

The judgement is about a **spoken turn**, and the modality changes what is cheap to collect:

- **Rating** — a thumb on the agent's reply, in the transcript view. Callers will not press it; the operator reviewing recordings will.
- **Preference** — the same caller question, answered twice under different sampling settings, played back side by side. Both sides get the identical retrieved context and the identical transcript, so the judgement is about generation rather than about which side heard the question better.
- **Correction** — the reply the agent *should* have given. On a voice product this is the most valuable signal, because the failures are usually policy failures rather than fluency failures.

Voice-specific rule: **judge on the transcript, train on the transcript.** Do not put audio in the training set. It is special-category data under GDPR Art. 9 in several readings, it carries consent obligations the text does not, and the LLM never saw it anyway — it saw ASR output.

7a. LoRA adapters: where they earn their place here

LoRA is already in this project's improvement loop as the vehicle for DPO. It earns a second, larger role here: **one base engine, many personas.**

Test this: per-tenant voice personas on one GPU. A voice product is usually sold to several clients at once, each wanting a different persona, tone and policy set. The naive answer is a deployment per client, which multiplies the GPU bill by the customer count. The right answer is one base engine plus a small adapter per tenant, selected per request.

Approach	GPU cost for 20 tenants	Switching cost
One deployment per tenant	20×	n/a
Prompt-only differentiation	1×	free, but weak — tone drifts and policy adherence is inconsistent
Base engine + per-tenant LoRA	**1×**	adapter swap per request

The TensorRT-LLM caveat, and state it plainly. Engines here are compiled ahead of time, so LoRA support has to be built into the engine (`--lora_plugin`, with `max_lora_rank` and the adapter slots declared at build time). You cannot add an adapter to an engine that was not built for one. Decide the maximum rank and slot count before the build, and put that constraint in the architecture graph — it is exactly the kind of engineering detail a technical buyer uses to judge whether you have actually run this in production.

Measure before believing. Adapter application is not free at inference time. Benchmark p95 TTFT with adapters active against the base engine, at realistic concurrency and with adapters actually being swapped between requests. On a sub-800 ms turn budget, a 30 ms regression is 4% of the budget and it must be a deliberate choice rather than a surprise.

---

8. The release gate for this project

`00-shared-requirements.md` §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	`lm_eval` (instruction following) + held-out spoken-support suite scored on policy adherence and correct escalation	Instruction following is the tightest bound; it predicts whether the agent stays inside supplied policy
Performance	Is it still fast?	`GuideLLM` against Triton at a realistic concurrent-call rate	**p95 TTFT is the gate that matters** — it sits directly inside the 800 ms turn budget, so a 20% TTFT regression is a product failure, not a slow-down

Also re-measure **word error rate through the full ASR → LLM → TTS loop**, not on clean text. Compression damage shows up differently when the input is already noisy from ASR.

---

9. Monitoring and KPIs

The headline chart is a stacked turn-latency budget, p50/p95/p99, split by stage. Everything else is secondary.

Section	Metrics
Quality	Task completion rate, escalation rate and reasons, ASR WER, interruption rate (users talking over the agent — the honest proxy for "does it feel slow")
Performance	Turn latency by stage, LLM TTFT and inter-token latency, tokens/sec, KV cache utilisation, in-flight batch occupancy, concurrent calls per GPU
Security	Prompt injection attempts (spoken injection is real — "ignore your instructions" said aloud), auth failures, rate-limit trips, 4xx/5xx
Audit	Call records with consent capture, retention state, redaction applied
Improvement (RLHF)	Turn approval rate, corrections captured, challenger win rate in A/B playback, preference pairs awaiting the next training run

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

Autoscale on queue depth or active-call count, never CPU. Alert when p95 turn latency crosses 800 ms, and when the interruption rate rises — the latter usually moves before users complain.

---

10. Security, audit and compliance

Voice adds obligations text does not have:

- **Recording consent must be captured and logged before audio is retained.** In many jurisdictions this is a legal precondition, not a nicety.
- **Voice is biometric data** under GDPR Art. 9 in several readings. Treat raw audio as special-category: minimise retention, encrypt with a separate key, and support erasure. Prefer discarding audio and keeping only the redacted transcript.
- **Spoken prompt injection** is a live attack surface. Detect and count it on the transcript, exactly as a text system would.
- Transcripts are PII-redacted on the write path.

---

11. Deployment

Kubernetes. GPU node pool with L40S or H100 — FP8 requires Hopper or Ada; on Ampere the engine falls back to INT4 and the latency target needs re-measuring.

The engine build is a **separate, versioned artifact** from the application image. It is large, slow to produce and specific to a GPU architecture; coupling it to application deploys would make every code change a multi-gigabyte push. Ship it as its own image, staged into the serving pod by an init container.

Warm minimum replicas of at least one. A cold TensorRT engine load is minutes, and a voice call cannot wait.

---

12. Deliverables

- Working browser voice interface with barge-in
- Engine build pipeline with FP8 quantization and a quality gate
- All three frontend tabs
- Grafana dashboards: turn-latency budget and product KPIs

- Helm charts and infrastructure as code
- Load test producing **measured** turn latency at concurrency
- README with the real numbers, not projections

Acceptance: 1. p95 turn latency under 800 ms at 10 concurrent calls, measured 2. Barge-in cancels generation within 200 ms of detected speech 3. Quality gate fails when a deliberately over-quantized engine is supplied 4. Architecture tab renders and the guided tour runs with the backend stopped

---

13. Out of scope

Telephony (SIP/PSTN) integration — browser WebRTC only. Speaker diarisation. Multilingual, beyond confirming the model does not break on accented English. Voice cloning, which carries consent problems disproportionate to a demo.